



Your AI Provider Will Fail. Design for It.

Your AI Provider Will Fail. Design for It.

The demo worked



Nobody wrote down what it was allowed to cost. Nobody wrote down what it should do if the call fails. Those questions weren't skipped on purpose. They weren't on any list anybody was working from.

And now this is on your desk

None of that is a model problem. All of it is an architecture problem.

1

A p99 nobody can defend

The response time depends on how long an answer happens to be.

2

A bill that moves with words

It varies with how many words a customer's conversation contained. No other dependency in your stack does that.

3

An on-call rotation that cannot tell

Down and confidently wrong look the same from the pager.

This is a dependency

The whole toolkit still works.

Timeouts

Retries

Tiers

Degradation

Blast radius

You don't need a new discipline. You need to apply the one you already know to something that behaves worse than usual.

Four properties that are worse

1

It is non-deterministic

The same input does not give the same output. Every test assertion you want to use stops working. That difference is normal behavior, not a bug you can file.

2

It is rate-limited by size, not just volume

Every other dependency counts your calls. This one counts your calls and how big each one is. Add context to a prompt and you get throttled without your traffic changing.

3

It is priced per use

Your bill scales with success. A feature that gets popular gets expensive on exactly the day you want to celebrate.

4

It can be wrong while it runs successfully

A 200, quickly, with an answer that is dead wrong. Every other dependency tells you when it failed. This one won't always know.

The feature's tier is not the dependency's tier

The feature

- Has a tier. Tier 1, Tier 2, whatever it is.
- Somebody assigned it deliberately.
- It sits on your service tier list.

The model underneath

- Almost certainly has no tier at all.
- Availability is usually committed.
- Response time often isn't.
- Answer quality never is.

A Tier 1 feature standing on a best-effort dependency is a decision. Most teams made it without noticing.

Six questions to ask about any AI dependency

- 1 Service Tier**

Have you given this dependency a criticality tier at all?
Do you have processes that know and manage the service's status?
- 2 Degradation**

What does the feature do when the model is down?
Is the feature simply turned off or disabled, or is there a greater impact to the operation of your application as a whole? How serious is a model outage?
- 3 Latency**

How long is the call allowed to take?
What happens when it takes longer?
- 4 Cost**

What is the ceiling on what this call may cost?
What's the highest you are willing to pay? What happens when you hit those limits?
- 5 Wrongness**

How would you know the answer was wrong?
- 6 Blast radius**

What can it reach, and what can it change?
Is sensitive PII data available to the model? Can the AI model change, update, or delete data? What external systems can it influence? What can it tell your customers? Can it make commitments on your behalf?

Six questions to ask about any AI dependency

- 1 Service Tier** Have you given this dependency a criticality tier at all?
- 2 Degradation** What does the feature do when the model is down?
- 3 Latency** How long is the call allowed to take?
- 4 Cost** What is the ceiling on what this call may cost?
- 5 Wrongness** How would you know the answer was wrong?
- 6 Blast radius** What can it reach, and what can it change?

Six questions to ask about any AI dependency

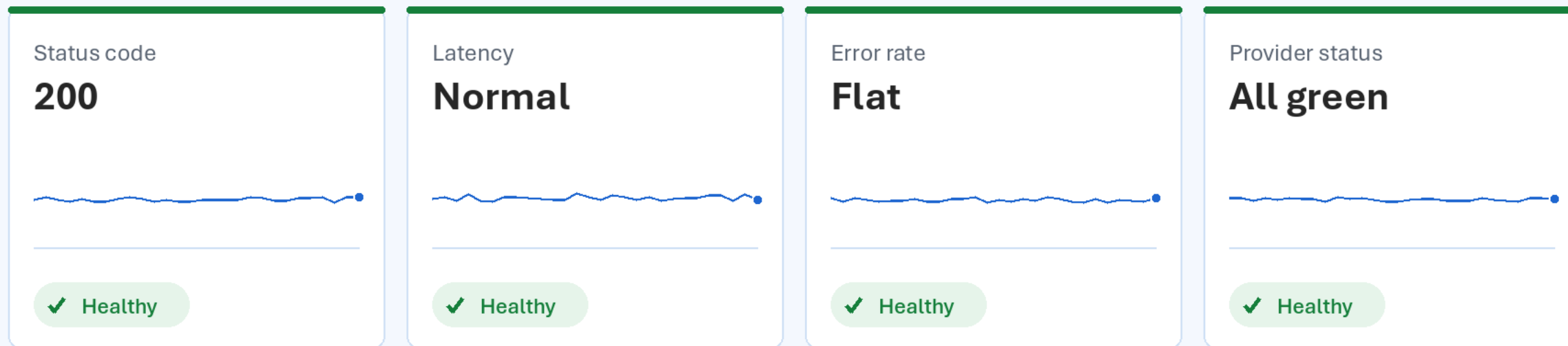
- 1 Service Tier** Have you given this dependency a criticality tier at all?
- 2 Degradation** What does the feature do when the model is down?
- 3 Latency** How long is the call allowed to take?
- 4 Cost** What is the ceiling on what this call may cost?
- 5 Wrongness** How would you know the answer was wrong?
- 6 Blast radius** What can it reach, and what can it change?

Six questions to ask about any AI dependency

5 **Wrongness**

How would you know the answer was wrong?

Question five: green dashboard, broken feature



And your customer still got a wrong answer.

Your monitoring was designed for a dependency that is either up or down. This failure mode is neither.

Question five: the test you can't write

What you had

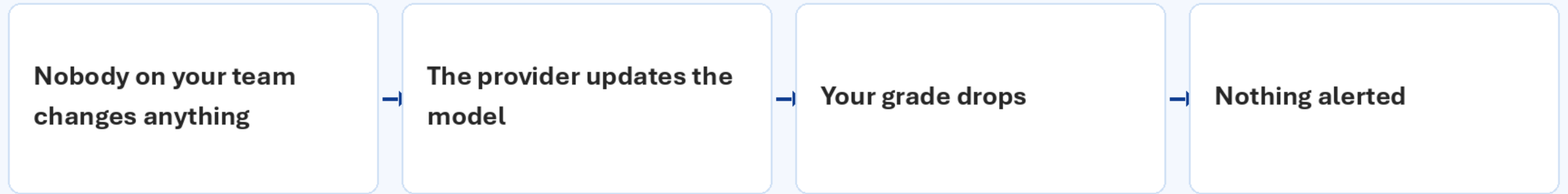
- assertEquals, and everything built on it.
- Regression tests. CI.
- A release gate that was pass or fail.
- A yes or a no.

What you get

- An evaluation harness.
- Graded examples you run like tests and read like a metric.
- A number. A percentile. A grade.
- Somebody has to make a judgement call.

Your release process was not built around “is a 75% quality response good enough?”

Question five: drift



Pinning to a model version helps, when the provider offers it. But versions get retired and the pin has to keep moving. And your prompts were tuned against a model that no longer exists. Silent degradation of a dependency you don't control, with no version number changing on either side.

Detecting drift: a fixed set of prompts, graded on a schedule

Not glamorous, not perfect, and far cheaper than a customer telling you.

1

Pick reference prompts out of real production traffic

Include the ones that went really badly. Easy prompts grade beautifully forever and tell you nothing.

2

Keep the job cheap and run it often

A grading run that takes an hour but happens once a quarter tells you nothing about the change that shipped on Tuesday.

Question five: how would you find out today?

Usually the answer is a customer complaint.

By the time a complaint gets back to you and is acted on, the wrong answers have been going out for a long time, to a lot of customers. And a grade on a schedule won't catch the one ticket that named the wrong account this morning. Grading tells you the feature runs at ninety-five percent. It doesn't tell you which five percent failed.

Three ways to watch live traffic

1

Sample and review

Pull a slice of real responses every day and have a person read them. Manual, and the only one that catches the failure nobody predicted.

2

Cross-check against a source of truth

If the model summarized an invoice, you still have the invoice. Compare the two numbers and alert on the mismatch. Usually automatable.

3

Ask the same question twice

Send the same request again and compare. When a model disagrees with itself, that is your signal. Costs an extra call, so save it for the answers that matter.

Who gets paged?

Down

- Has an owner.
- Has an alert.
- Has a runbook.
- Has a name attached to it.

Wrongness

- Usually has none of those.
- The runbook says check the provider status page.
- The status page is green.
- The engineer holding the pager has no next step.

Decide who owns wrongness before you need to know, and write the wrong-answer runbook now.

Six questions to ask about any AI dependency

- 1 Service Tier** Have you given this dependency a criticality tier at all?
- 2 Degradation** What does the feature do when the model is down?
- 3 Latency** How long is the call allowed to take?
- 4 Cost** What is the ceiling on what this call may cost?
- 5 Wrongness** How would you know the answer was wrong?
- 6 Blast radius** What can it reach, and what can it change?

Six questions to ask about any AI dependency

- 1 Service Tier** Have you given this dependency a criticality tier at all?
- 2 Degradation** What does the feature do when the model is down?
- 3 Latency** How long is the call allowed to take?
- 4 Cost** What is the ceiling on what this call may cost?
- 5 Wrongness** How would you know the answer was wrong?
- 6 Blast radius** What can it reach, and what can it change?

Six questions to ask about any AI dependency

6 **Blast radius**

What can it reach, and what can it change?

Question six: what did you hand it?

- **Draw the data flow** What data leaves your system, where does it land, who keeps it, and for how long.
- **Read it out loud** Which of those lines could you say to a customer without going and looking first? Almost everybody has one where the answer is uncomfortable.
- **Do they train on it?** Sitting in a contract somebody signed.
- **What country does inference run in?** Also that contract. The person who wired up the API call didn't read it.

Prompt injection

Why it is worse than SQL injection

- The model can't tell your instructions from your data.
- To a model it is all just text.
- Anything you put in the prompt as data can be read as an instruction.

Where it comes from

- A support ticket description.
- A resume.
- A fetched web page.
- A database row somebody else wrote.

You can't filter your way out. Every filter is a guess about what an attack looks like. Treat it as a permissions problem instead.

Design the boundary, not the filter

1

Give the model its own identity

Its own principal with its own scopes. Not the user's whole session, and not a shared service account with everything already on it.

2

Split read from write

A model that reads your database and one that writes to it are two different risk profiles. Most features only need the first.

3

Put something deterministic in front of anything irreversible

Sending money, deleting records, emailing a customer.
Rules-checked before the model is allowed to act.

4

Decide which actions need a person

A human in the loop is the cheapest control you have on an action you cannot take back.

Question six: grant narrowly, and grant again

- **Your blast radius is what you granted** The list of tools, credentials and scopes you handed over. It has nothing to do with what the model asks for.
- **Review the tool list** Every tool is a verb the model can use, and the list grows every sprint because adding one is a one-line change nobody reviews.
- **Ask the worst case out loud** If somebody got this model to call this tool with any arguments they wanted, what is the damage? If you don't like the answer, it needs a check in front of it.

Six questions to ask about any AI dependency

- 1 Service Tier** Have you given this dependency a criticality tier at all?
- 2 Degradation** What does the feature do when the model is down?
- 3 Latency** How long is the call allowed to take?
- 4 Cost** What is the ceiling on what this call may cost?
- 5 Wrongness** How would you know the answer was wrong?
- 6 Blast radius** What can it reach, and what can it change?

Six questions to ask about any AI dependency

1 Service Tier Have you given this dependency a criticality tier at all?

2 Degradation What does the feature do when the model is down?

3 Latency How long is the call allowed to take?

4 Cost What is the ceiling on what this call may cost?

5 Wrongness How would you know the answer was wrong?

6 Blast radius What can it reach, and what can it change?

Questions one and two: tier and degradation

Question one is the tier: put the provider on the list next to the feature. Question two is what the feature does when the call fails, and there are four answers.

1

Serve a cached answer

The last good response, clearly enough that nobody mistakes it for fresh.

2

Take a deterministic path

Rules, a lookup, a template. Whatever the feature used before a model replaced it.

3

Narrow the scope, or hide the feature

On purpose, and visibly.

4

Fail, and say so out loud

The honest option, and sometimes the right one.

Question three: latency

Pick the budget. An actual number, in milliseconds, written down.

Then enforce it, and decide what the feature does when a call blows past it. Most teams do none of that. They measure what the demo did and treat that as the number. Demo prompts are short and demo answers are short. Production is neither.

Question four: cost

- **Cost is a failure mode** A runaway bill takes a feature offline exactly the way an outage does. Somebody decides to turn it off.
- **And that person didn't design it** So the feature comes back later with the ceiling still unset.
- **Set the ceiling first** Per request. Per user. Per day. Decide where the limit lives and who may raise it.
- **At the ceiling, get cheaper** Pick one of the four degradation answers in advance. Don't get switched off by somebody who was never in the design review.

Monday: six lines in your architecture decision record

Open your ADR template. Add six lines.

Tier

Degradation

Latency

Cost

Wrongness

Blast radius

About ten minutes. After that every AI dependency your team adds gets asked these six questions by your process, instead of by whoever happened to remember.

Six questions to ask about any AI dependency

- 1 Service Tier**

Have you given this dependency a criticality tier at all?
Do you have processes that know and manage the service's status?
- 2 Degradation**

What does the feature do when the model is down?
Is the feature simply turned off or disabled, or is there a greater impact to the operation of your application as a whole? How serious is a model outage?
- 3 Latency**

How long is the call allowed to take?
What happens when it takes longer?
- 4 Cost**

What is the ceiling on what this call may cost?
What's the highest you are willing to pay? What happens when you hit those limits?
- 5 Wrongness**

How would you know the answer was wrong?
- 6 Blast radius**

What can it reach, and what can it change?
Is sensitive PII data available to the model? Can the AI model change, update, or delete data? What external systems can it influence? What can it tell your customers? Can it make commitments on your behalf?